

Summer Overview



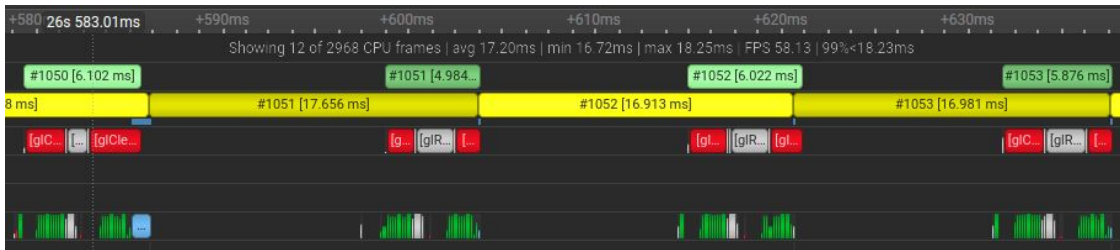
Debugging with Swastik:)

Problems

1. Sim performance bottleneck
 - a. CPU/GPU
 - b. Suitable machine?
2. ML model pipeline
 - a. Schedule various ML models efficiently on the GPU
3. Perceptions Step Time
 - a. Perception freezes
 - b. Step time reduction
4. Constant monitoring of step time metrics
5. Slam speed up

SIM Performance Blockers

- Isolated sim test with image rendering and cutting (headless, single core)
 - Dozer does a 20 sec sequence of tracking, cutting deep, and then recovering
- Nsight gpu profiling
 - `nsys profile --trace="" --output=<dir>/<name-of-the-file> cmd <executable> <args>`
- `perf record -g -call-graph dwarf`
 - Dwarf stores instance of stack memory
 - Call trace shows
 - Agx::physics and other computations do not offload to GPU
 - `perf report --stdio`
- Sim (AIM internal) function - `processDepthImage` (6% of total utilization)
 - Max CPU utilization outside Algorix
- GPU handles image rendering



```
--1.15%--(anonymous namespace)::ExpandConstraints::dispatch
--1.15%--agx::Physics::RigidBody::ExpandConstraints
--1.17%--agx::IndexLambdaKernel::execute
--1.17%--agx::IndexLambdaKernel::executeIndexRange
--1.11%--concurrent::execPreSystemAndUpdateValidFunc
--0.95%--agx::HighLevelConstraintImplementation::updateValid
--0.65%--agx::AttachmentPair::transform
--2.66%--agx::JobGroup::implementation
--1.96%--agx::JobGroup::implementation
--1.92%--agx::DirectSolverData::solve
--1.92%--agx::BoxedHybridPivotAlgorithm::solve
--1.91%--agx::BoxedMurtayAlgorithm::solve
--1.71%--agx::NLMixedCp::McpAlgorithm::solvePp
--1.65%--agx::Sabre::Sabre::modifyMatrix<double>
--0.62%--agx::Sabre::SabreFactor::partialFactor<double>
--0.52%--sabreFactorImpl_SSE_FUSED
--0.59%--agx::SolveJob<(anonymous namespace)::PrepareDirectSolvers>::implementation
--0.58%--agx::Physics::RigidBody::Solver::PrepareDirectSolvers
--1.68%--agx::RangeJob<(anonymous namespace)::ComputeNarrowPhaseContacts>::implementation
--1.68%--agx::Physics::Geometry::ComputeNarrowPhaseContacts
--1.47%--agx::Collide::ShapeCollider::calculateContacts
--0.55%--agx::Collide::collideBoxHesh
--0.55%--agx::Collide::GroupFooCollider<(unsigned char)>::calculateContacts
--0.55%--agx::Collide::ShapeCollider::calculateContacts
--0.54%--agx::Collide::CylinderBoxColliders::calculateContacts
--0.53%--agx::Collide::CylinderBoxCollisions::collideCylinderBox
--0.53%--agx::Collide::collideConvexConvex
--0.50%--btCjkEpaSolver2::penetration
--0.53%--agx::Collide::collideConvexConvex
--0.50%--btCjkEpaSolver2::penetration
--0.89%--agx::Physics::Geometry::SynchronizeTransforms
```

Samples: 97K of event 'cpu_core/cycles/', Event count (approx.): 131725299528, DSO: libnvidia-mlcore.so.535.129.03

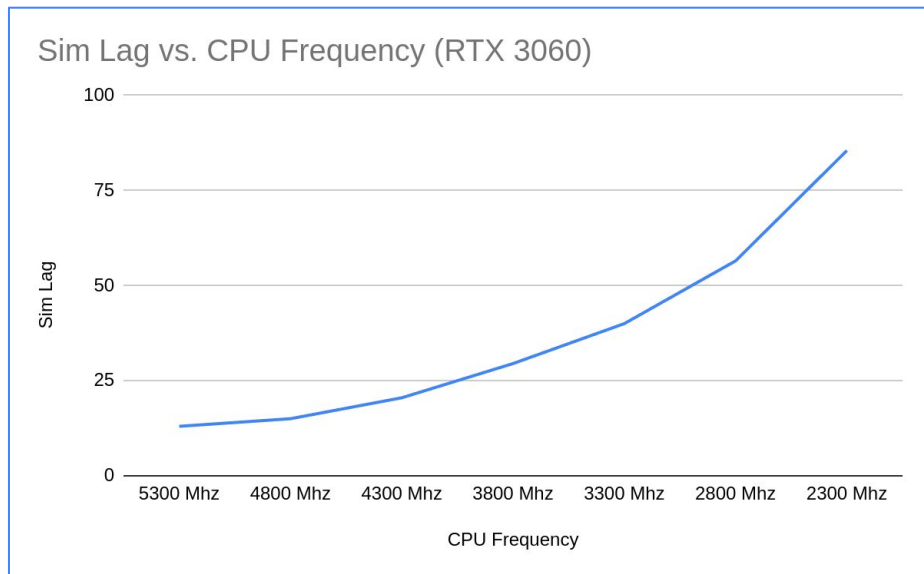
Children	Self	Command	Symbol
10.32%	0.00%	AgxSLn	[.] 0x00007fba9c593525
6.67%	0.00%	AgxSLn	[.] 0x00007fba9f9c0e0d
6.53%	0.00%	AgxSLn	[.] 00007fba9b536a1
6.51%	0.00%	AgxSLn	[.] 0x00007fba9b51b7c
6.50%	0.00%	AgxSLn	[.] 0x00007fba9f9c70a
6.50%	0.00%	AgxSLn	[.] 0x00007fba9f908cc0
5.88%	0.00%	AgxSLn	[.] 0x00007fba0245ae
5.49%	0.00%	AgxSLn	[.] 0x00007fba023c66
4.81%	0.00%	AgxSLn	[.] 0x00007fba092572
4.64%	0.00%	AgxSLn	[.] 0x00007fba0930b1
4.55%	0.00%	AgxSLn	[.] 0x00007fba08f773
3.35%	0.00%	AgxSLn	[.] 0x00007fba04dc41
2.89%	0.00%	AgxSLn	[.] 0x00007fba03f8701
2.89%	2.89%	AgxSLn	[.] 0x0000000019f8701
1.44%	1.44%	AgxSLn	[.] 0x0000000019f870c
1.44%	0.00%	AgxSLn	[.] 0x00007fba03f870c
1.25%	0.00%	AgxSLn	[.] 0x00007fba092609
1.12%	0.00%	AgxSLn	[.] 0x00007fba9f9b0d85
1.12%	1.12%	AgxSLn	[.] 0x0000000015b0d85
0.75%	0.75%	AgxSLn	[.] 0x0000000015b0d00
0.75%	0.00%	AgxSLn	[.] 0x00007fba9f9b0d00
0.74%	0.00%	AgxSLn	[.] 0x00007fba9f911038
0.71%	0.00%	AgxSLn	[.] 0x00007fba9f910f83
0.55%	0.55%	AgxSLn	[.] 0x0000000015b0d97
0.55%	0.00%	AgxSLn	[.] 0x00007fba9f9b0d97
0.51%	0.00%	AgxSLn	[.] 0x00007fba02495f
0.49%	0.49%	AgxSLn	[.] 0x0000000015b0d7c
0.49%	0.00%	AgxSLn	[.] 0x00007fba0327cb
0.47%	0.47%	AgxSLn	[.] 0x0000000015b0d9f
0.47%	0.00%	AgxSLn	[.] 0x00007fba9f9b0d9f
0.46%	0.46%	AgxSLn	[.] 0x0000000015b0d8f
0.46%	0.00%	AgxSLn	[.] 0x00007fba9f9b0d8f
0.46%	0.46%	AgxSLn	[.] 0x0000000015b0d78
0.46%	0.00%	AgxSLn	[.] 0x00007fba9f9b0d78
0.38%	0.38%	AgxSLn	[.] 0x0000000010327d9
0.38%	0.00%	AgxSLn	[.] 0x00007fba0327d9
0.37%	0.00%	AgxSLn	[.] 0x00007fba9d92240
0.36%	0.00%	AgxSLn	[.] 0x00007fba03c7edf
0.36%	0.00%	AgxSLn	[.] 0x00007fba9d90e0f
0.32%	0.00%	AgxSLn	[.] 0x00007fba9bc491d
0.32%	0.00%	AgxSLn	[.] 0x00007fba023c7c
0.31%	0.00%	AgxSLn	[.] 0x00007fba0460cc
0.31%	0.31%	AgxSLn	[.] 0x0000000010327e0
0.31%	0.00%	AgxSLn	[.] 0x00007fba0327e0
0.29%	0.00%	AgxSLn	[.] 0x00007fba0227fc
0.27%	0.00%	AgxSLn	[.] 0x00007fba03f86f6
0.27%	0.27%	AgxSLn	[.] 0x0000000019f86f6
0.26%	0.00%	AgxSLn	[.] 0x00007fba01aa25
0.25%	0.00%	AgxSLn	[.] 0x00007fba08f624
0.25%	0.00%	AgxSLn	[.] 0x00007fba9cb54d3
0.25%	0.00%	AgxSLn	[.] 0x00007fba9cb4f4c
0.24%	0.00%	AgxSLn	[.] 0x00007fba9cb092d
0.24%	0.00%	AgxSLn	[.] 0x00007fba9c9470e
0.24%	0.00%	AgxSLn	[.] 0x00007fba9c9c4d2
0.24%	0.00%	AgxSLn	[.] 0x00007fba9c9c70e
0.23%	0.00%	AgxSLn	[.] 0x00007fba9f78a92
0.22%	0.22%	AgxSLn	[.] 0x0000000015b0e01
0.22%	0.00%	AgxSLn	[.] 0x00007fba9f9b0e01
0.22%	0.00%	AgxSLn	[.] 0x00007fba011f7
0.22%	0.00%	AgxSLn	[.] 0x00007fba03c273
0.21%	0.21%	AgxSLn	[.] 0x0000000015b0f1
0.21%	0.00%	AgxSLn	[.] 0x00007fba9f9b0d1
0.21%	0.00%	AgxSLn	[.] 0x00007fba9f908c55
0.21%	0.00%	AgxSLn	[.] 0x00007fba0441e66
0.21%	0.00%	AgxSLn	[.] 0x00007fba9f9c0e4e
0.20%	0.00%	AgxSLn	[.] 0x00007fba9f9b0dd
0.20%	0.20%	AgxSLn	[.] 0x0000000015b0ddd
0.19%	0.00%	AgxSLn	[.] 0x00007fba0460b82
0.18%	0.00%	AgxSLn	[.] 0x00007fba0454131
0.18%	0.00%	AgxSLn	[.] 0x00007fba0450b0e

- Conclusion: Physics computation on CPU, Rendering on GPU
- Escalated sim lag was noticed at cutting
 - CPU

CPU Instruction cycles

- Cloud instances with higher RAM, comparable GPU's show worse performance
 - CPU clock frequency?
 - Experiment executes millions of instructions

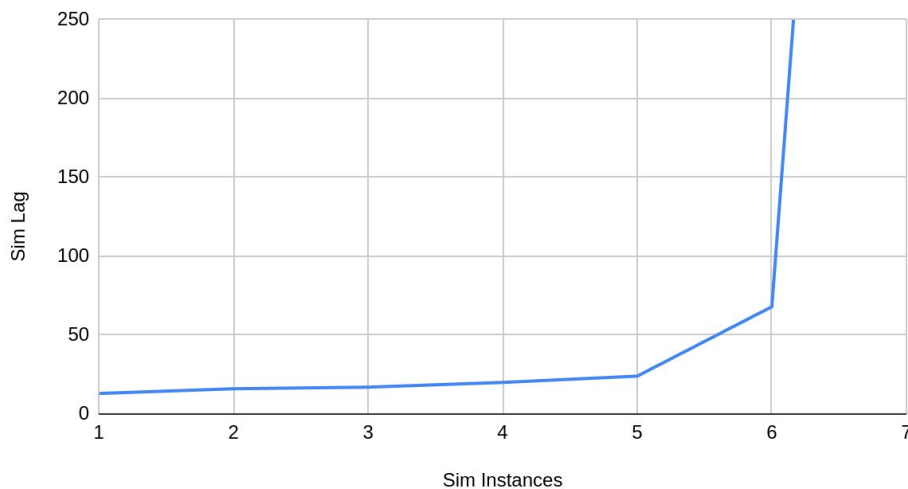
CPU Frequency	Sim Lag	Machines
3300 Mhz	91	A10G, G5
2200 Mhz	137	L4, Basic (4 cores)
2200 Mhz	135	A100, high intensity (8 cores)
5300 Mhz	13	RTX 3060
4800 Mhz	15	RTX 3060
4300 Mhz	20.5	RTX 3060
3800 Mhz	29.5	RTX 3060
3300 Mhz	40	RTX 3060
2800 Mhz	56.5	RTX 3060
2300 Mhz	85.5	RTX 3060



- `watch -n 0.5 "cat /proc/cpuinfo | grep -i mhz"`
- `cpupower frequency-set -u <value-in-khz>`

- Single sim instance is running at single core
- **Stalls on bus & GPU?**
 - Saturation Point (see Graph)
- Multiple cores do not help single instance
 - Iterative computation?

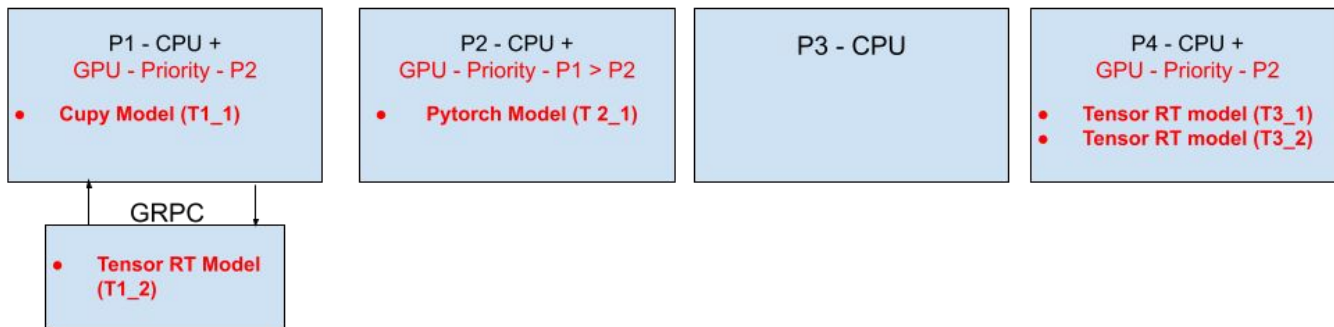
Sim Lag vs. Sim Instances



Take Away

- CPU cycles is the major bottleneck for sim
 - For cutting terrain
- Still have leverage on the GPU (multi-instance test)
- 5.3 Ghz provides perfect performance optimization
- RTX GPU's are preferred for efficient image rendering
- Best Resource:
 - Processor Clock - 5.3Ghz, Nvidia RTX 3060 <

ML Pipeline



- Problems
 - GPU resource contention?
 - Efficiently run different models on GPU
 - Reduce resource contention
 - Task priority
- Idea
 - Scheduler
 - Handle different user-gpu interaction libraries?
 - All cuda based tasks

GPU Pipeline

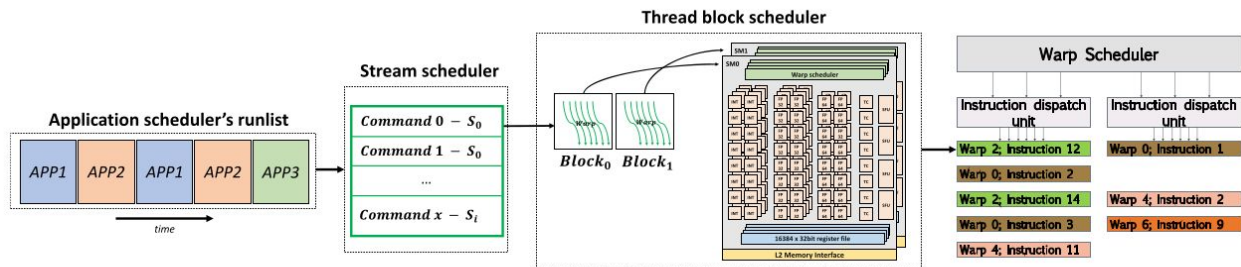


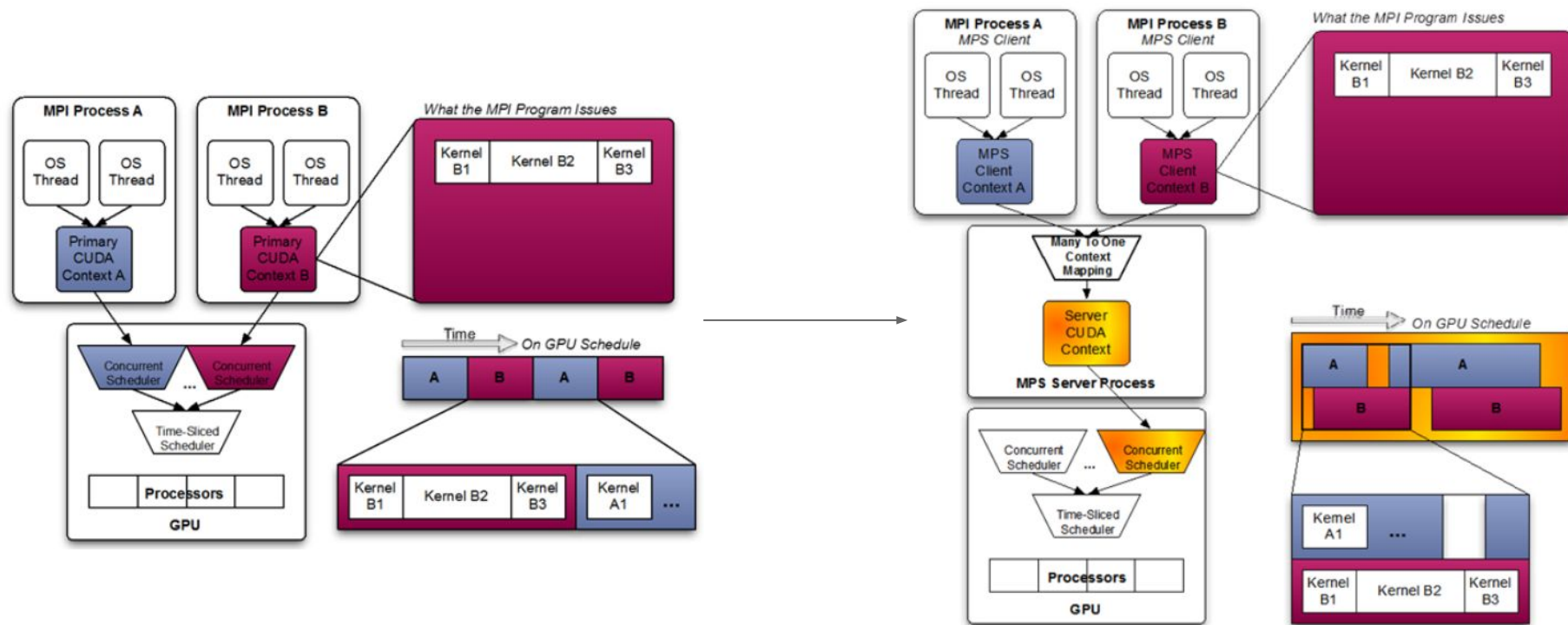
Figure 4: GPU scheduling hierarchy. Streams from S_0 to S_i belong to an application selected from the *runlist*.

LOOKING INSIDE A STREAMING MULTIPROCESSOR

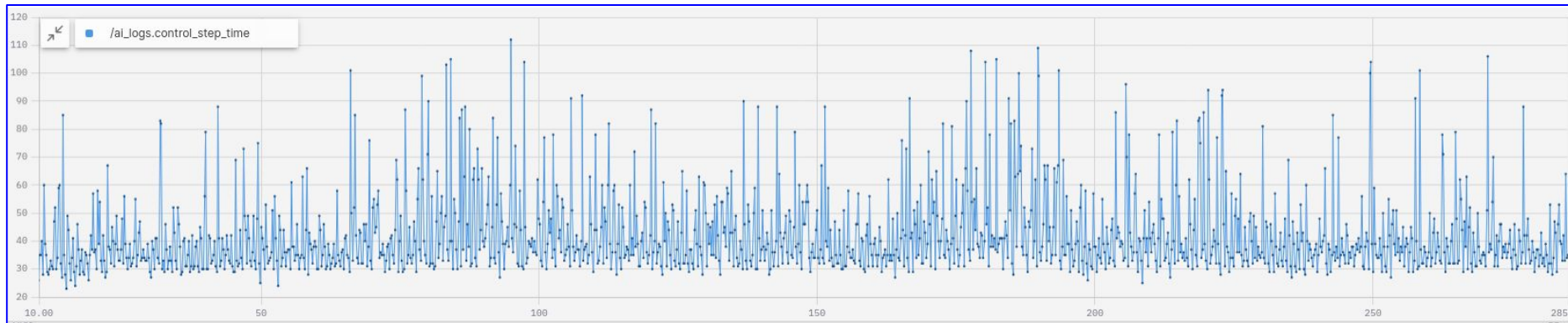
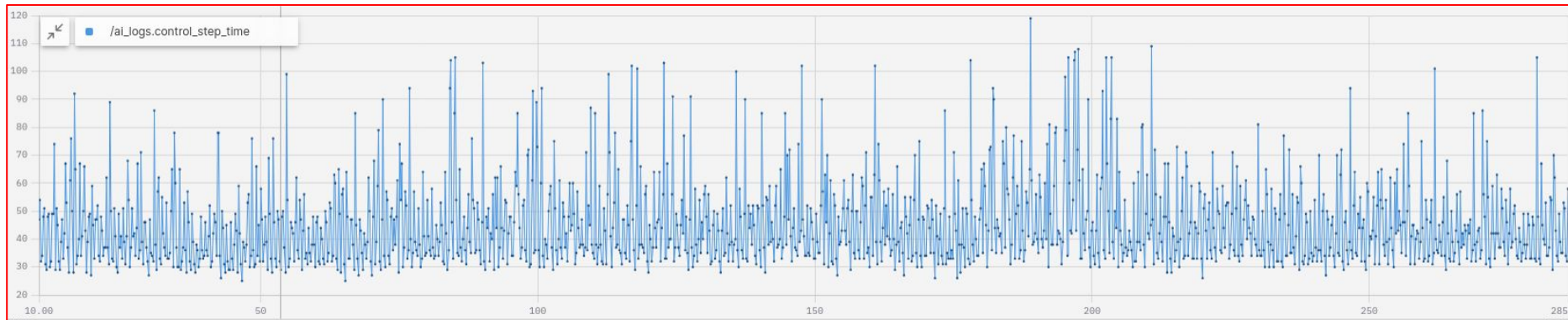
A100 SM Resources	
2048	Max threads per SM
32	Max blocks per SM
65,536	Total registers per SM
160 kB	Total shared memory in SM
32	Threads per warp
4	Concurrent warps active
64	FP32 cores per SM
32	FP64 cores per SM
192 kB	Max L1 cache size
90 GB/sec	Load bandwidth per SM
1410 MHz	GPU Boost Clock

- Multiple parallel processors within the GPU
- Scheduler over Nvidia Cuda?

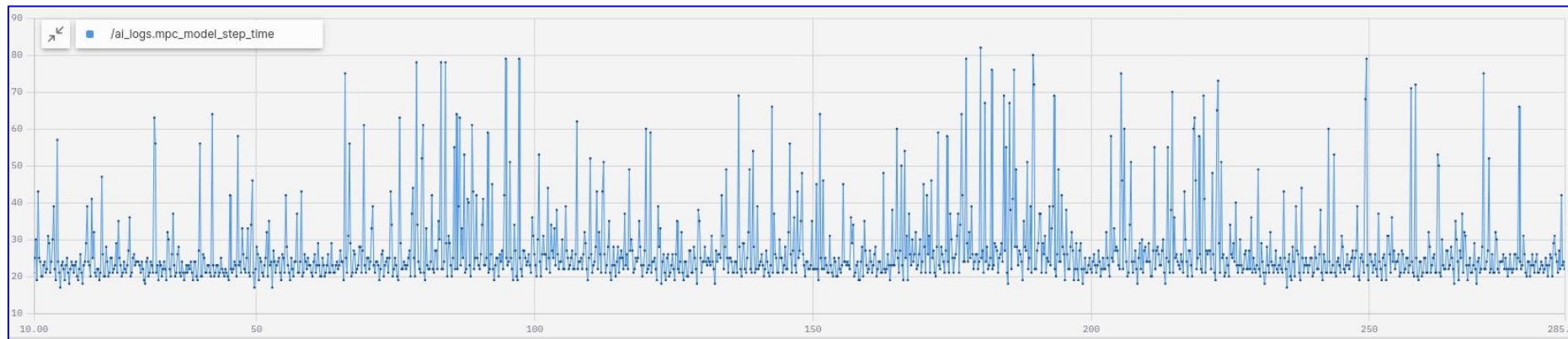
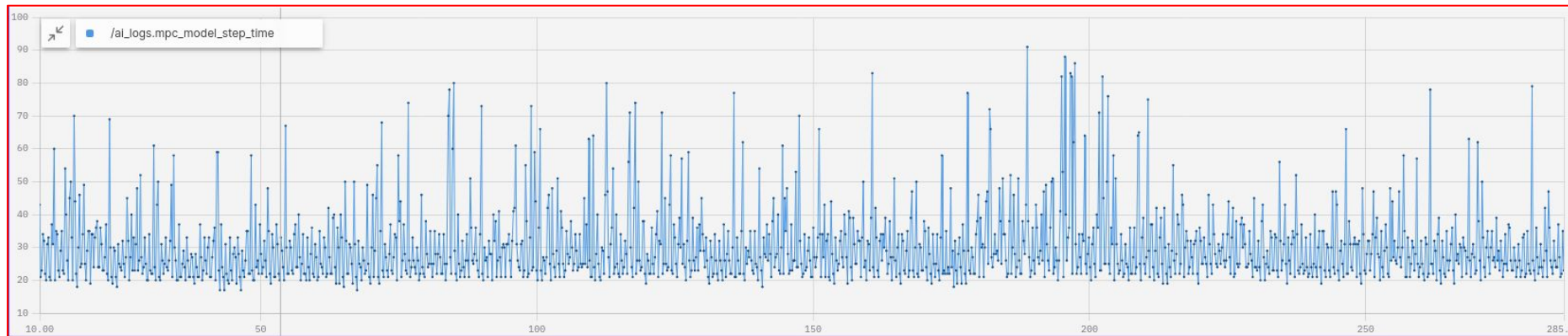
MPS - Multi-Processing Service (Model scheduling pipeline)



D10 MPC - Replay Logs (No MPS | MPS)

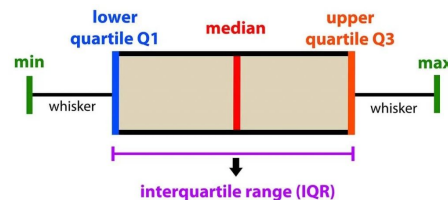
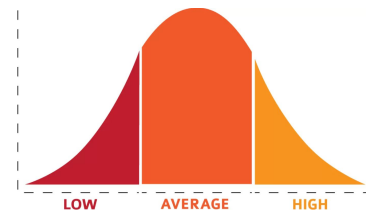


MPC model step time (No MPS | MPS)

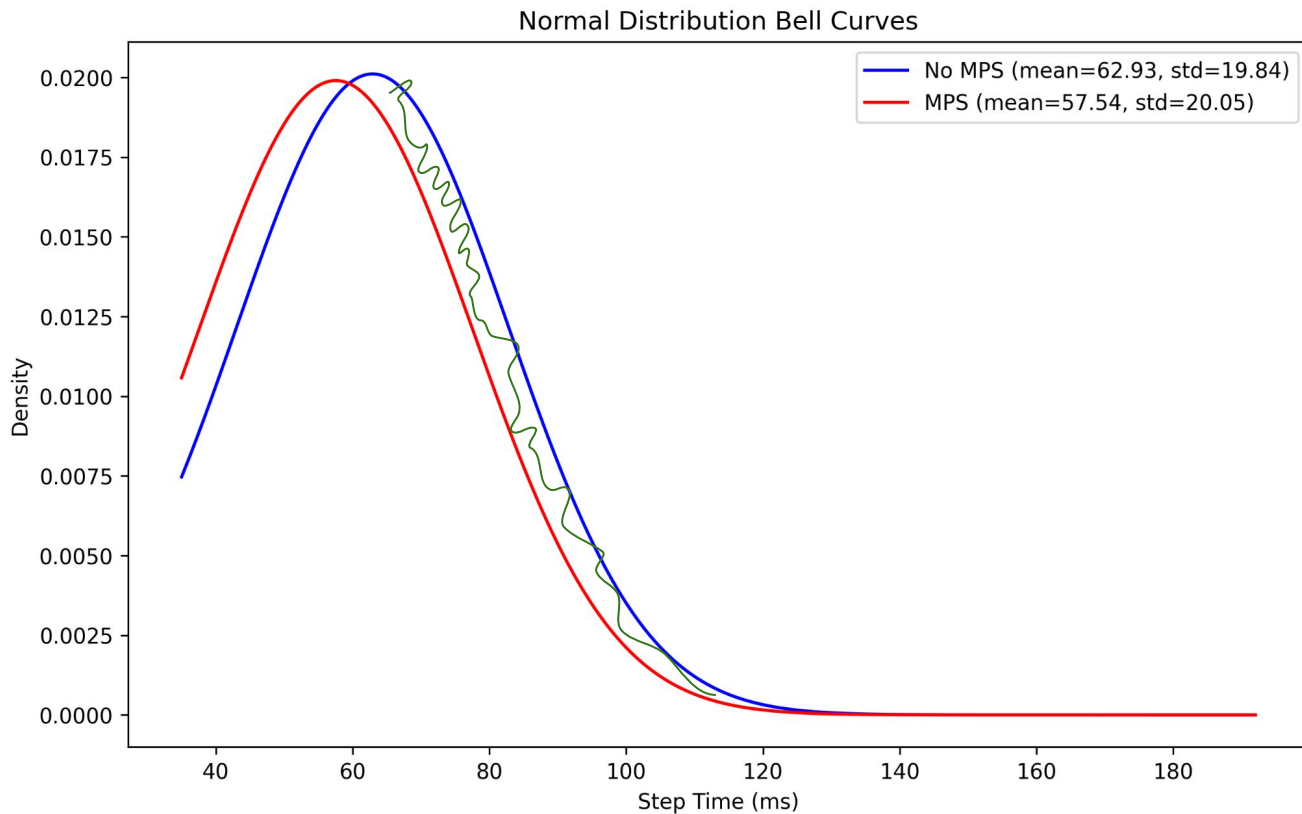


What would be an accurate analysis?

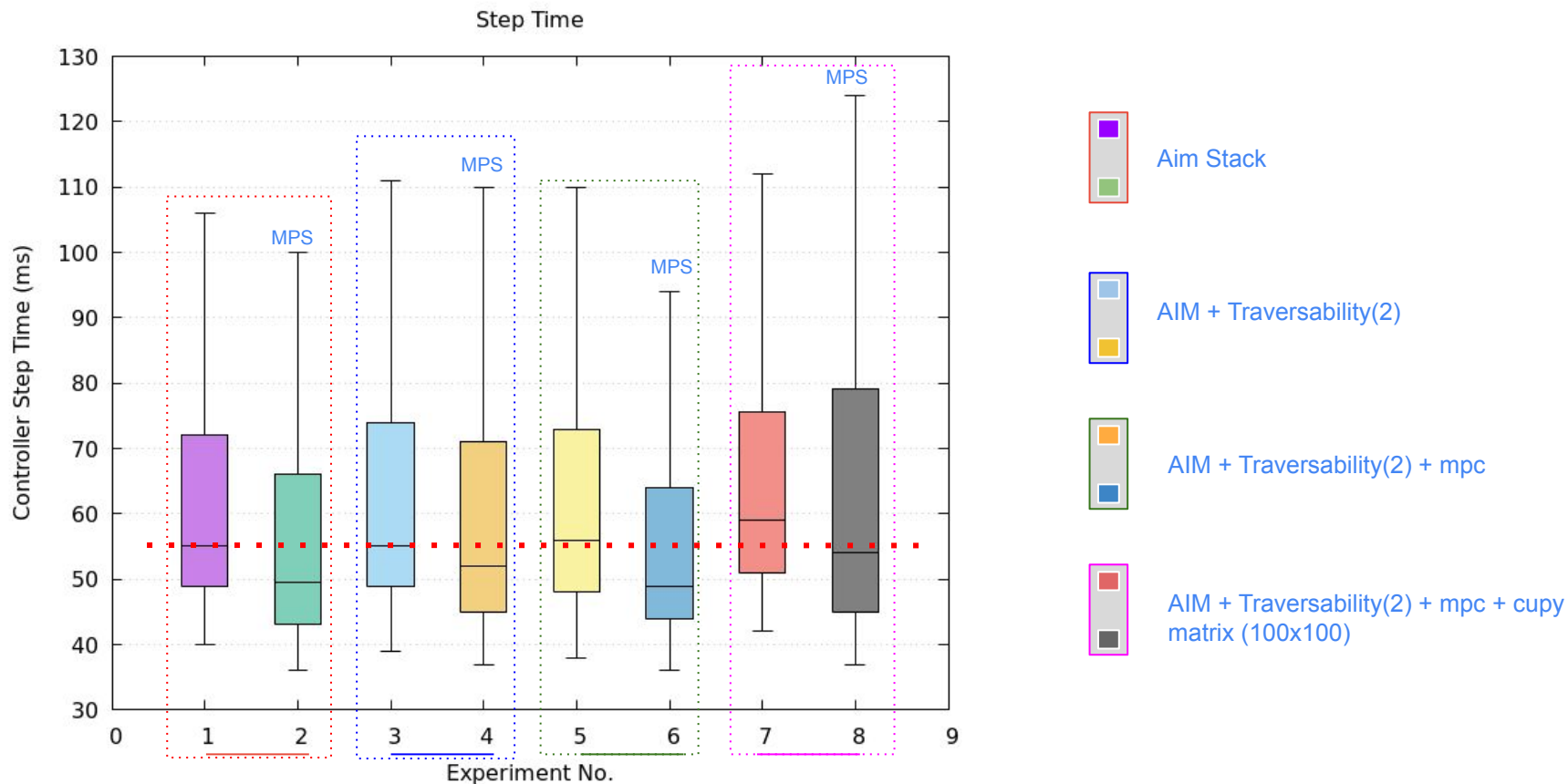
- Step time graphs are not an accurate metric (Include CPU Time)
 - Controller step time includes CPU operations which can result in a non-deterministic worst case time
 - Resulting in higher averages
 - MPC step time would be a **little** more accurate eliminating some of the CPU time
- **distribution of step time** (*More values with reduced step time due to parallel GPU tasks*)
 - Expectation: **Normal distribution**
 - Bell curve shifted towards the left with MPS (**not necessarily the peaks**)
 - Imagine a right skewed distribution
 - **Box Plot:**
 - lower median
 - data points are lower in value
 - compared to the group with the higher median
 - Left shifted curve (irrespective of the skew of distribution)



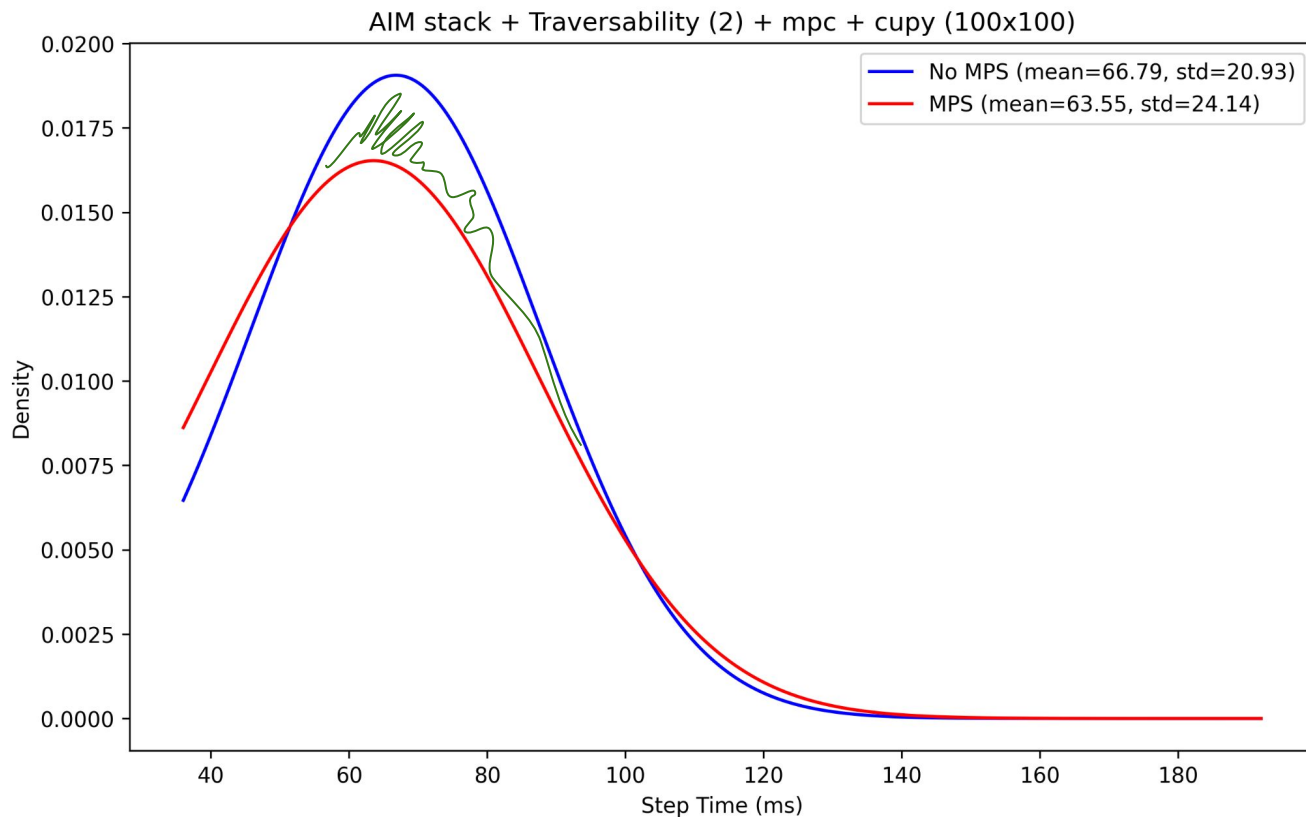
Experimental Evaluation on SIL (D10 MPC)

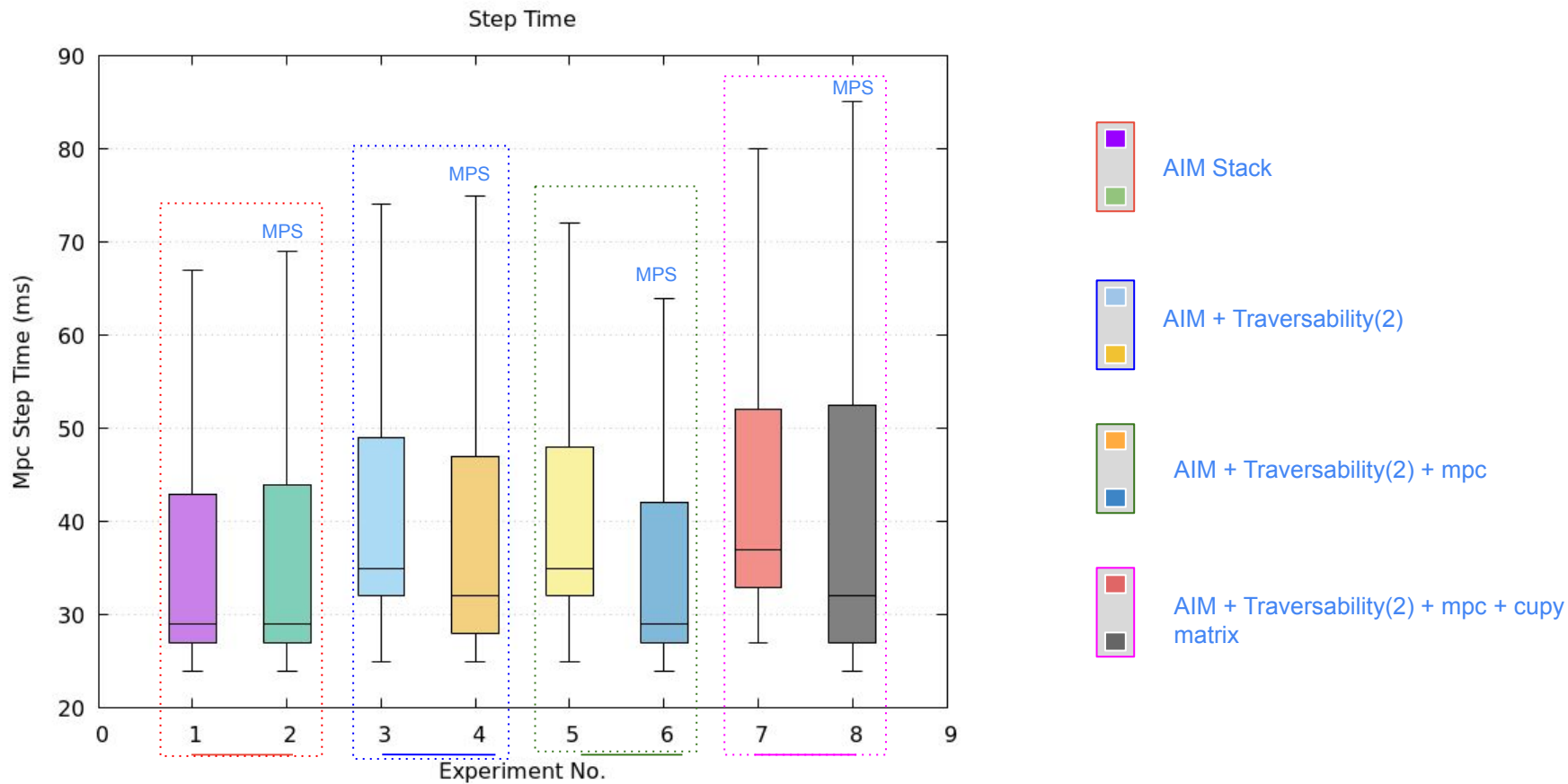


Experimental Evaluation on SIL (D10 MPC)



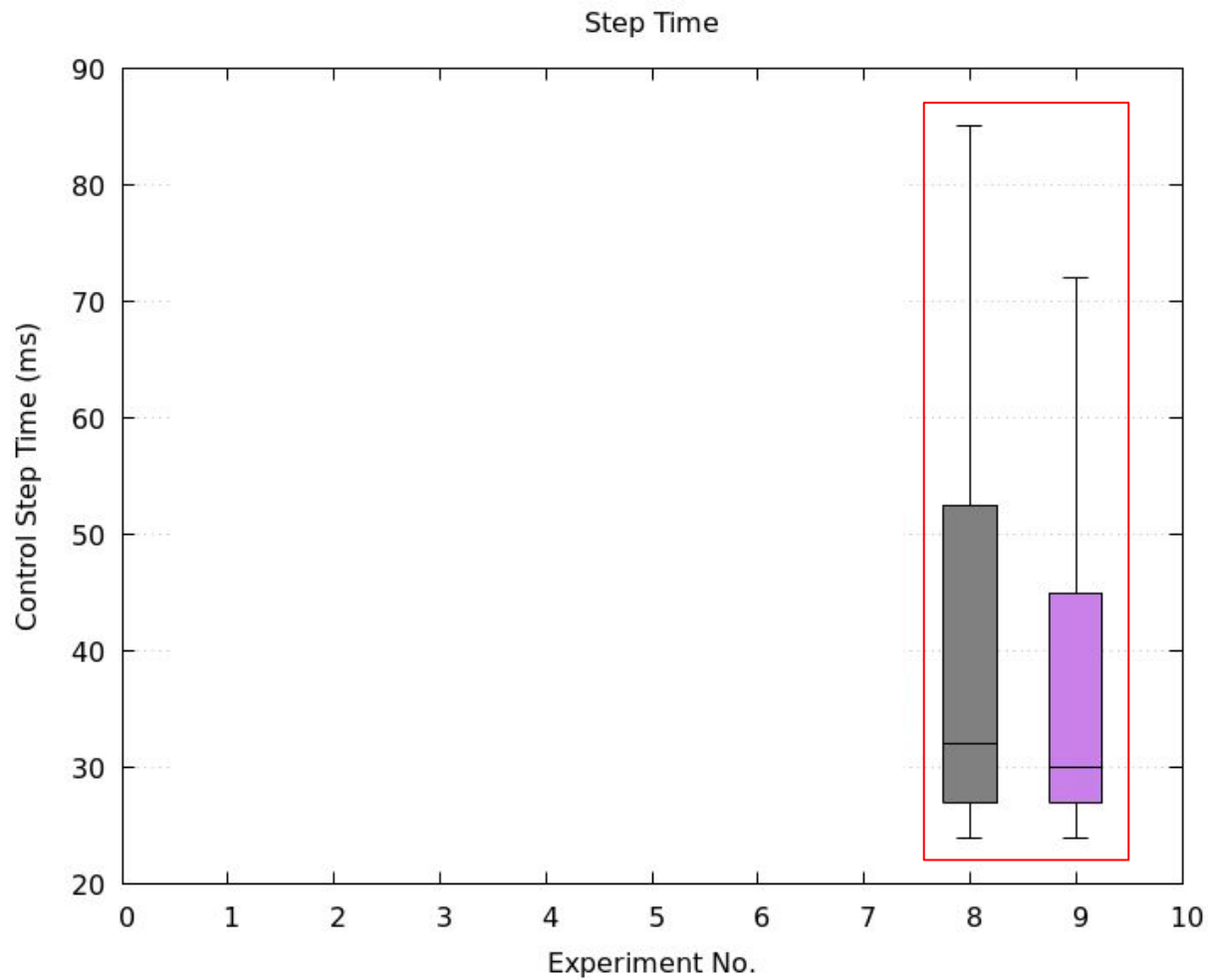
Normal distribution for the max load





Task Priorities

- Modify server priority for upcoming tasks
 - `echo `set_active_thread_percentage <PID> <percentage>` | nvidia-cuda-mps-control`
 - Task scheduled post this run at the set priority
 - Priorities are (0 and 1)
 - 0 - default priority
 - 1- lower than default



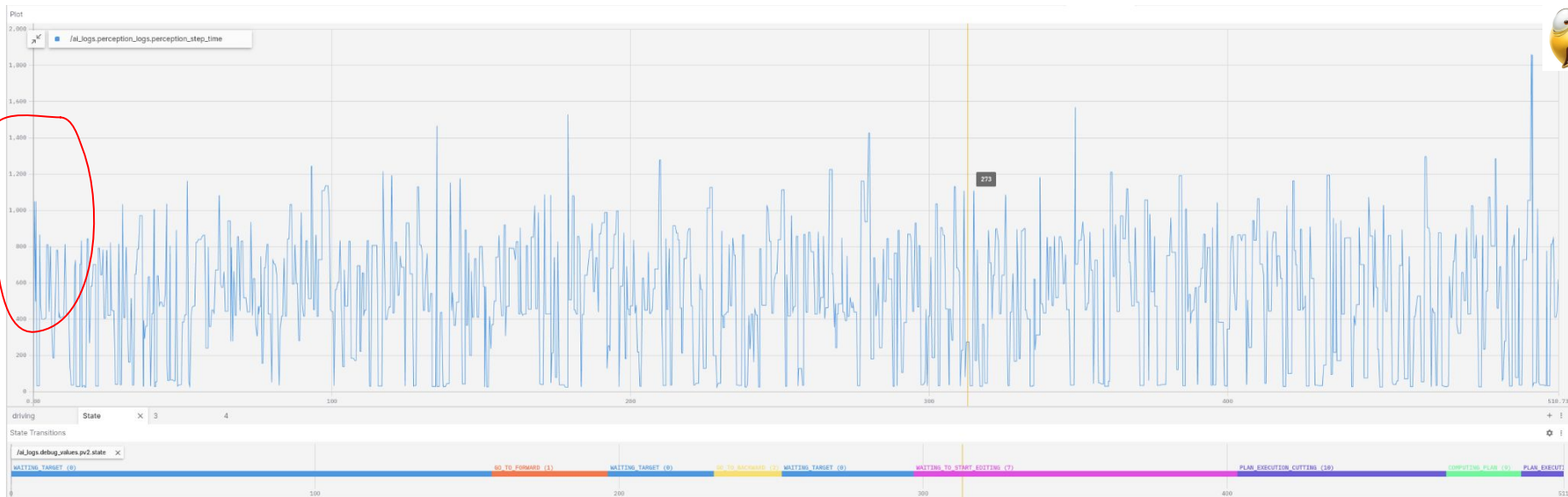
Last run
with MPS
priority

MPS Take Away

- Benefits of task parallelism on GPU
 - Kernel execution of one task can at min. execute memory copy of another
- Provides the required pipeline for parallel execution of various models
- Allows additional models on the stack without much overhead
- **MPS Saturation:** only adds an overhead due to scheduler abstraction
 - Cupy matrix of 10Kx10K (4 parallel processes) running along the mpc
 - 85% GPU memory exhausted (nvidia-smi - **17270MiB / 20475MiB**)
 - MPS - MPC avg. step time - 600ms
 - No MPS - MPC avg. step time - 860 ms

Perception Step Time Freeze

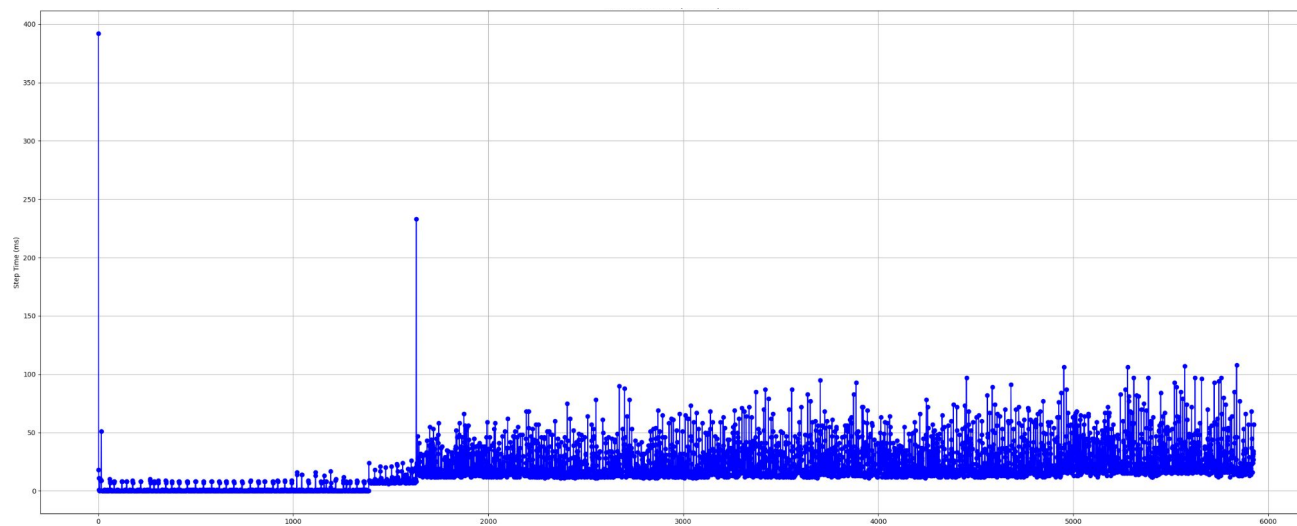
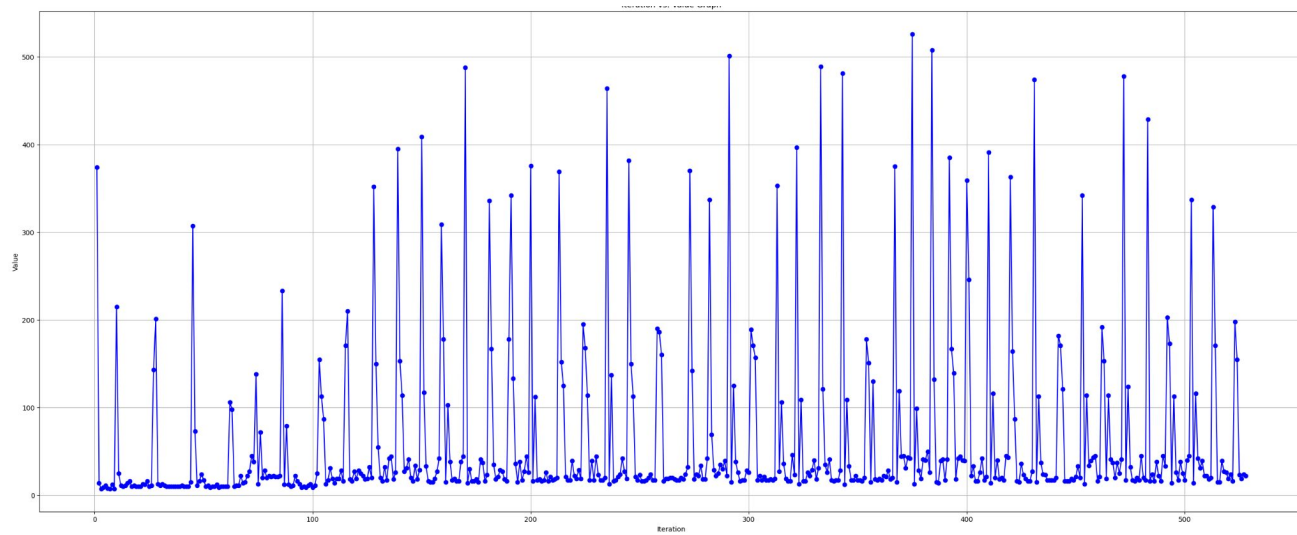
- Problem (MTI task on D5)



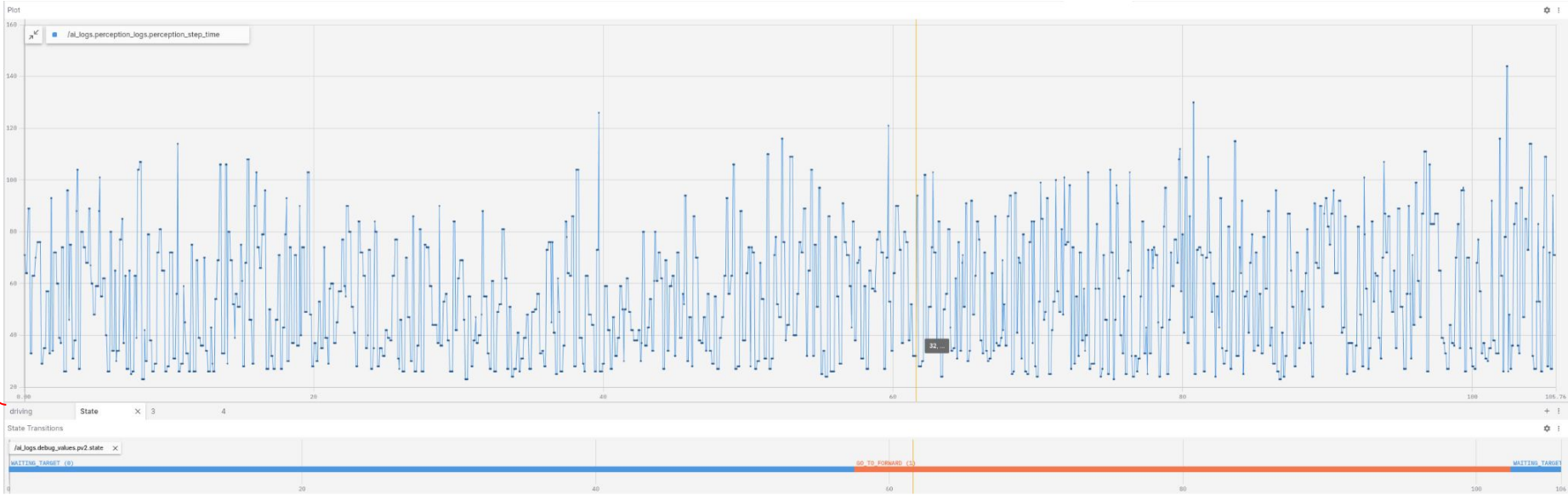
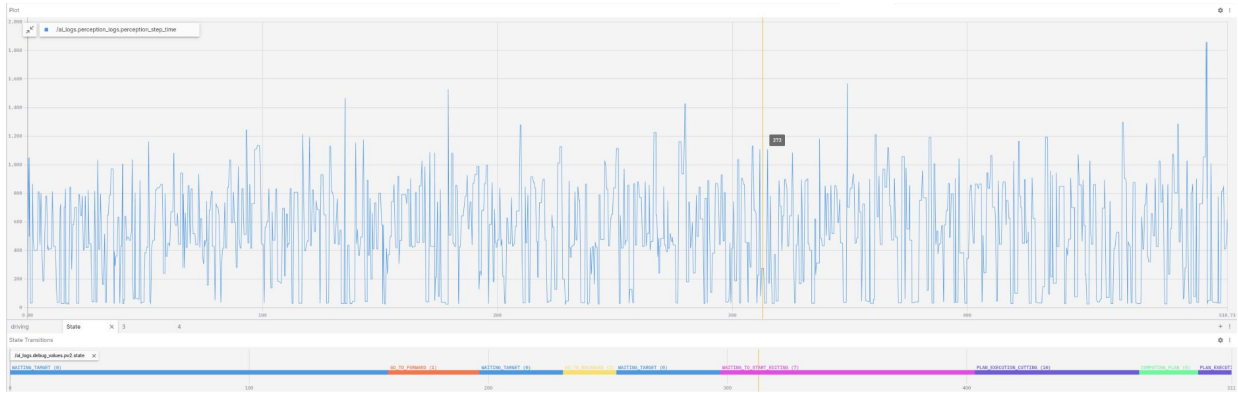
Possible Issues

- Code bugs
 - Step time freeze consistent within the function (AIM perception - `add_lidar_points`)
 - **Inconsistent within itself**
 - Series of cupy operations (GPU contention?)
- Resource contention
 - **Issues only with traversability**
 - Traversability running in parallel does not portray the same (No GPU contention?)
 - Perception and traversability within the same cuda context
 - Different streams for parallel GPU resource utilization ↓
- Memory I/O/ sync issues
 - GPU memory I/O on the profiler did not show any unrealistic scenario (**no crazy queuing delays**)
 - Grpc I/O
 - Traversability model step time is low
 - Encoding/casting/serialization
 - Grpc Serialization ↑

Vola!



D5



Future Scope for Step Time Improvement

- GPU shared memory
 - L2 cache shared within a streaming multiprocessor
 - Arrays can be explicitly shared within a cuda kernel into a shared memory
 - Cache level speedup
- Stream synchronization
 - Each process runs as a separate context within the same stream by default
 - Default synchronizations
 - No need for explicit synchronization
- Shared Memory

Shared Memory (Impl.)

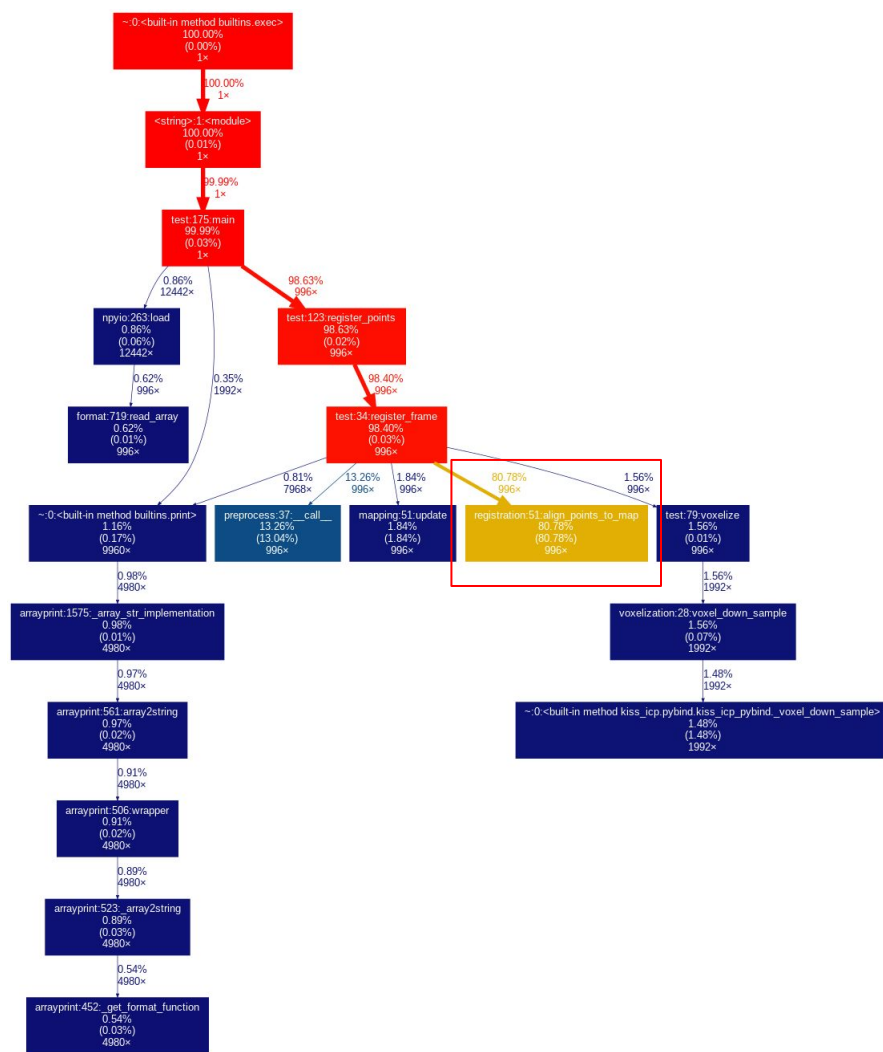
- Shared memory takes 1.4 microseconds to transfer FullStateAtT message
- GRPC - 210 microseconds
 - Approx. 150x speedup using shared memory
 - not including the serialization time:)
- Posix-ipc
 - Allows locking based shared memory
 - Test implementation
 - Verifies correct resource utilization for single writer / multiple readers
 - **Writer waits for all readers to complete**

Step Time Metrics on Grafana (#3178)

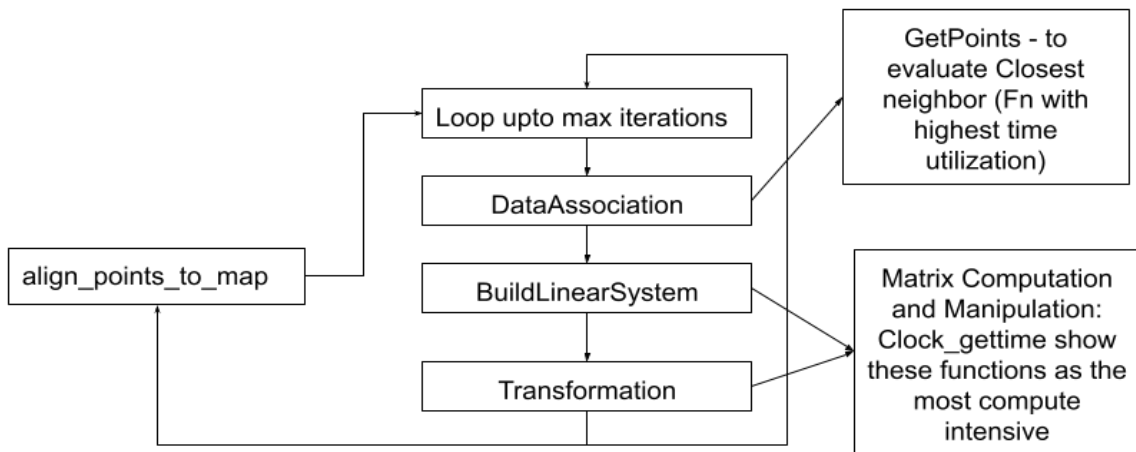


Slam Profiling

- Python cprofile
 - Complete profiling (no sampling)
- Perf
 - Time Utilization + Call Graph
- Timing via code instrumentation



SLAM Speedup



1. GetPoints has more calls with each call having nanoseconds level precision (40% of the total time)
 - a. Cache Improvement?

- GPU offloading
 - OpenMP static vs dynamic sched.
 - Dynamic scheduling shows better step time
 - Iterative algorithm
 - Higher memory I/O with GPU
- Memoization
 - Matrix on each iteration is different
- Compiler optimization flags
 - O3
- Rust Compiler
 - Much slower with rust

14:16:50-07:00:509 README.md test.py									
(b5) swastik@swastik:~/kiss-icp\$ cp base_profile_result.py ../kiss-icp-rs/									
(b5) swastik@swastik:~/kiss-icp\$ python base_profile_result.py									
Wed Jun 12 14:23:53 2024 /home/swastik/kiss-icp/base_results_new.txt									
1401403 function calls (135849 primitive calls) in 43.035 seconds									
Ordered by: cumulative time									
ncalls tottime pcall cntime percall filename:lineno(function)									
1	0.000	0.000	43.035	43.035	0.000	0.000	43.035	43.035	built-in method builtins.exec
1	0.000	0.000	43.035	43.035	0.000	0.000	43.035	43.035	__init__(module)
1	0.013	0.013	43.035	43.035	0.000	0.000	43.035	43.035	test.py:175(main)
575	0.000	0.000	42.513	0.074	0.000	0.000	42.513	0.074	test.py:123(register_points)
424	0.000	0.000	42.411	0.000	0.000	0.000	42.411	0.000	test.py:246(format_array_header)
575	34.550	0.000	34.550	0.000	0.000	0.000	34.550	0.000	registration.py:51(align_points_to_map)
375	3.727	0.000	0.174	0.000	0.000	0.000	3.727	0.000	preprocess.py:37(cut_...)
375	0.000	0.001	0.166	0.000	0.000	0.000	3.727	0.001	mapping.py:51(update_...)
575	0.003	0.000	0.094	0.001	0.000	0.000	3.727	0.001	test.py:79(voxelize)
1	0.030	0.000	0.081	0.000	0.000	0.000	3.727	0.000	voxelization.py:28(voxel_down_sample)
1159	0.000	0.001	0.069	0.001	0.000	0.000	3.727	0.001	built-in method kiss_icp.pybind.kiss_icp.pybind_voxel_down_sample
5745	0.079	0.000	0.540	0.000	0.000	0.000	3.727	0.000	built-in method builtins.print
5745	0.000	0.000	0.540	0.000	0.000	0.000	3.727	0.000	arrayprint.py:152(array_print_implementation)
2872	0.010	0.000	0.457	0.000	0.000	0.000	3.727	0.000	arrayprint.py:561(array2string)
2872	0.000	0.000	0.438	0.000	0.000	0.000	3.727	0.000	arrayprint.py:586(wrapper)
2872	0.012	0.000	0.419	0.000	0.000	0.000	3.727	0.000	arrayprint.py:523(array2string)
7087	0.029	0.000	0.268	0.000	0.000	0.000	3.727	0.000	npysio.py:263(load)
2872	0.012	0.000	0.258	0.000	0.000	0.000	3.727	0.000	arrayprint.py:452(get_format_function)
2872	0.004	0.000	0.211	0.000	0.000	0.000	3.727	0.000	arrayprint.py:411(lambda)
2872	0.000	0.000	0.207	0.000	0.000	0.000	3.727	0.000	arrayprint.py:907(_init_)
2872	0.018	0.000	0.190	0.000	0.000	0.000	3.727	0.000	arrayprint.py:934(fill_format)
575	0.004	0.000	0.158	0.000	0.000	0.000	3.727	0.000	format.py:75(read_array)
2872	0.003	0.000	0.157	0.000	0.000	0.000	3.727	0.000	arrayprint.py:780(formatArray)
40796	0.082	0.000	0.140	0.000	0.000	0.000	3.727	0.000	arrayprint.py:780(recursive)
575	0.000	0.000	0.110	0.000	0.000	0.000	3.727	0.000	format.py:582(read_array_header)
575	0.000	0.000	0.085	0.000	0.000	0.000	3.727	0.000	built-in method kiss_icp.pybind.kiss_icp.pybind_preprocess
575	0.000	0.000	0.077	0.000	0.000	0.000	3.727	0.000	format.py:547(fill_header)
1148	0.001	0.000	0.074	0.000	0.000	0.000	3.727	0.000	from_data.py:7(array)
1148	0.072	0.000	0.072	0.000	0.000	0.000	3.727	0.000	built-in method cupy_core.core.array
6087	0.000	0.000	0.070	0.000	0.000	0.000	3.727	0.000	built-in method numpy.core.multiarray.unath.implement_array_function
1	0.000	0.000	0.068	0.000	0.000	0.000	3.727	0.000	test.py:105(_init_)
1	0.000	0.000	0.067	0.000	0.000	0.000	3.727	0.000	basic.py:96(cpus)
12075	0.019	0.000	0.054	0.000	0.000	0.000	3.727	0.000	tokenize.py:429(tokenize)
31028	0.031	0.000	0.046	0.000	0.000	0.000	3.727	0.000	arrayprint.py:1018(_call_)
1	0.004	0.000	0.044	0.000	0.000	0.000	3.727	0.000	basic.py:96(cpus)
31028	0.010	0.000	0.041	0.000	0.000	0.000	3.727	0.000	arrayprint.py:753(extend_line_pretty)
1147	0.002	0.000	0.037	0.000	0.000	0.000	3.727	0.000	_array_function__internals>177((inv)
575	0.030	0.000	0.037	0.000	0.000	0.000	3.727	0.000	built-in method numpy.frontline
23044	0.000	0.000	0.035	0.000	0.000	0.000	3.727	0.000	arrayprint.py:989(generator)
1147	0.023	0.000	0.034	0.000	0.000	0.000	3.727	0.000	linalg.py:483(inv)
7088	0.030	0.000	0.030	0.000	0.000	0.000	3.727	0.000	built-in method io.open
2870	0.003	0.000	0.028	0.000	0.000	0.000	3.727	0.000	_array_function__internals>177(ana)
5740	0.007	0.000	0.025	0.000	0.000	0.000	3.727	0.000	fromnumeric.py:60(get_prediction)
575	0.000	0.000	0.024	0.000	0.000	0.000	3.727	0.000	test.py:92(get_prediction_model)
5740	0.010	0.000	0.024	0.000	0.000	0.000	3.727	0.000	ufunc_config.py:332(enter)
14638	0.011	0.000	0.023	0.000	0.000	0.000	3.727	0.000	built-in method builtins.max
23044	0.011	0.000	0.023	0.000	0.000	0.000	3.727	0.000	arrayprint.py:984(generator)
575	0.002	0.000	0.022	0.000	0.000	0.000	3.727	0.000	utils.py:907(safe_eval)
1	0.000	0.000	0.022	0.000	0.000	0.000	3.727	0.000	method 'fill' of 'cupy_core.core._ndarray_base' objects
2870	0.004	0.000	0.022	0.000	0.000	0.000	3.727	0.000	fromnumeric.py:2677(ana)
14638	0.022	0.000	0.022	0.000	0.000	0.000	3.727	0.000	built-in method numpy.core.multiarray.unath.dragond_positional
1	0.000	0.000	0.022	0.000	0.000	0.000	3.727	0.000	compiler.py:216(compile_module_with_cache)
1	0.000	0.000	0.022	0.000	0.000	0.000	3.727	0.000	compiler.py:582(compile_with_cache_cuda)
1	0.000	0.000	0.021	0.000	0.000	0.000	3.727	0.000	compiler.py:438(preprocess)
1	0.000	0.000	0.021	0.000	0.000	0.000	3.727	0.000	compiler.py:604(compile)
1	0.021	0.021	0.021	0.021	0.000	0.000	3.727	0.000	built-in method cupy_backends.cuda.libs.nvrtc.compileProgram
575	0.002	0.000	0.021	0.000	0.000	0.000	3.727	0.000	ast.py:54(literal_eval)
2870	0.003	0.000	0.020	0.000	0.000	0.000	3.727	0.000	ufunc_config.py:429(enter_)
31028	0.015	0.000	0.020	0.000	0.000	0.000	3.727	0.000	arrayprint.py:739(extend_line)
6315	0.010	0.000	0.019	0.000	0.000	0.000	3.727	0.000	method 'reduce' of 'numpy.ufunc' objects
352491	0.017	0.000	0.017	0.000	0.000	0.000	3.727	0.000	built-in method builtins.len
10056	0.004	0.000	0.016	0.000	0.000	0.000	3.727	0.000	arrayprint.py:963(generator)

- 0.06 sec for kiss-icp
- 1.95 sec for kiss-icp rust

Significant altering within the slam code is needed for any speed ups. This could be task specific.